



1. BDD EMPLEADA PARA ADMINISTRAR EL NEGOCIO DE LA EMPRESA iLap

1. BDD EMPLEADA PARA ADMINISTRAR EL NEGOCIO DE LA EMPRESA iLap.....	1
1.1. Creación y configuración de contenedores.....	2
1.1.1. Configurar contenedores.....	4
1.2. Creación de objetos en la BDD.....	8
1.2.1. Reglas generales.....	8
1.2.2. Creación de usuarios.....	8
Ejemplo.....	9
Ejemplo.....	9
1.2.3. Creación de ligas.....	10
Ejemplo.....	10
1.2.4. Creación de fragmentos.....	11
Ejemplo.....	1

1.1. Creación y configuración de contenedores

Como se mencionó en la parte 1, la base de datos distribuida del proyecto requiere de la simulación de 4 nodos. Cada integrante aporta 2 PDBs a través de una CDB instalada y configurada en un contenedor Docker. Por lo anterior, existirán 2 contenedores Docker por equipo. La correcta operación de la base de datos distribuida requiere conectividad entre estos 2 contenedores. Para implementar este requerimiento, seguir el siguiente procedimiento.

- A partir del contenedor Docker empleado para realizar las prácticas del curso se creará una imagen.
- A partir de la imagen generada, se crearán 2 nuevos contenedores Docker con su respectiva CDB. Estos 2 contenedores serán configurados para poder formar una BD distribuida formada por 2 servidores con 2 PDBs.

Cada integrante del equipo deberá realizar las siguientes actividades en su equipo.

- A. Cerrar o salir del contenedor en caso de ser necesario. Ejecutar el siguiente comando en la máquina host para mostrar los datos de los contenedores existentes

```
jorge@pc-bdx-jrc:~$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	NAMES
f92427112b02	8d0218cd023e	"bash"	10 months ago	Exited (127) 2 seconds ago	c1-bdd-jrc

- B. De la salida anterior, observar el valor del nombre del contenedor. Emplearlo para crear una nueva imagen. Ajustar el nombre del contenedor según corresponda.

```
docker commit c1-bdd-<iniciales> bdd-<iniciales>:1.0
```

- En el ejemplo anterior se crea una nueva imagen llamada `bdd-<iniciales>` con el tag `1.0` a partir del contenedor `c1-bdd-<iniciales>`. Ejemplo real:

```
docker commit c1-bdd-jrc bdd-jrc:1.0
```

- C. El siguiente paso es crear una subred de Docker que permita la comunicación entre los contenedores que serán generados más adelante. Ejecutar la siguiente instrucción

```
docker network create --subnet=172.20.0.0/16 bdd-proy-net
```

- D. Considerar los siguientes nombres para crear los contenedores

- `c1-bdd-proy-<iniciales_integrante1>`
- `c2-bdd-proy-<iniciales_integrante2>`

E. Al momento de crear cada contenedor es importante especificar la IP y nombre de host. Considerar los siguientes nombres de host y las IPs para emplear en los contenedores.

- `h1-bdd-proy-<iniciales_integrante1>.fi.unam` , IP: `172.20.0.21`
- `h2-bdd-proy-<iniciales_integrante2>.fi.unam` , IP: `172.20.0.22`

F. Crear 2 nuevos contenedores a partir de las imágenes anteriores. Ajustar los valores resaltados según corresponda.

```
#contenedor que representa al integrante 1 (jrc)
docker run -i -t \
-v /tmp/.X11-unix:/tmp/.X11-unix \
-v ${UNAM_HOME}:${UNAM_HOME} \
--name c1-bdd-proy-jrc \
--hostname h1-bdd-proy-jrc.fi.unam \
--expose 1521 \
--shm-size=2gb \
--net=bdd-proy-net \
--ip 172.20.0.21 \
--add-host h2-bdd-proy-arc.fi.unam:172.20.0.22 \
-e DISPLAY=${DISPLAY} bdd-jrc:1.0 bash
```

```
#contenedor que representa al integrante 2 (arc)
docker run -i -t \
-v /tmp/.X11-unix:/tmp/.X11-unix \
-v ${UNAM_HOME}:${UNAM_HOME} \
--name c2-bdd-proy-arc \
--hostname h2-bdd-proy-arc.fi.unam \
--expose 1521 \
--shm-size=2gb \
--net=bdd-proy-net \
--ip 172.20.0.22 \
--add-host h1-bdd-proy-jrc.fi.unam:172.20.0.21 \
-e DISPLAY=${DISPLAY} bdd-jrc:1.0 bash
```

- Observar la configuración de los siguientes parámetros
 - En contenedor 1: `--add-host h2-bdd-proy-arc.fi.unam:172.20.0.22 \`
 - En contenedor 2: `--add-host h1-bdd-proy-jrc.fi.unam:172.20.0.21 \`

- Esta opción agrega una entrada adicional al archivo `/etc/hosts` en cada contenedor, permite resolver la IP del otro contenedor a partir de su nombre de host.
- Esta configuración es muy importante para poder comunicarse entre contenedores empleando los nombres de hosts. Recordar que todos los archivos de configuración de red en Oracle hacen uso de nombres de host. Ajustar las iniciales correctamente.

G. Agregar alias en la máquina host para acceder fácilmente a los contenedores.

```
alias dockerBddProy1='docker start c1-bdd-proy-jrc && docker attach c1-bdd-proy-jrc'
alias dockerBddProy1T='docker exec -it c1-bdd-proy-jrc bash'

alias dockerBddProy2='docker start c2-bdd-proy-arc && docker attach c2-bdd-proy-arc'
alias dockerBddProy2T='docker exec -it c2-bdd-proy-arc bash'
```

- En el ejemplo anterior se emplea el sufijo `1` y `2` que hace referencia al contenedor de cada integrante. Se puede emplear otra nomenclatura si así se prefiere.

1.1.1. Configurar contenedores

- El valor del parámetro `--add-host` va a generar la siguiente configuración en el archivo `/etc/hosts` para ambos contenedores.
- Abrir 2 terminales en la máquina host, acceder a cada contenedor y mostrar el contenido del archivo `/etc/hosts`. Revisar que se tenga una configuración bidireccional similar a la siguiente:

```
#configuración en contenedor 1, se agregó la segunda línea
172.20.0.21      h1-bdd-proy-jrc.fi.unam h1-bdd-proy-jrc
172.20.0.22      h2-bdd-proy-arc.fi.unam

#configuración en contenedor 2, se agregó la segunda línea
172.20.0.22      h2-bdd-proy-arc.fi.unam h2-bdd-proy-arc
172.20.0.21      h1-bdd-proy-jrc.fi.unam
```

- Comprobar la configuración anterior realizando `ping` hacia el otro contenedor empleando el hostname:

```
#ping desde contenedor 1 hacia el contenedor 2
[root@h1-bdd-proy-jrc /]# ping h2-bdd-proy-arc.fi.unam
PING h2-bdd-proy-arc.fi.unam (172.20.0.22) 56(84) bytes of data.
64 bytes from h2-bdd-proy-arc.fi.unam (172.20.0.22): icmp_seq=1 ttl=64 time=0.088 ms
64 bytes from h2-bdd-proy-arc.fi.unam (172.20.0.22): icmp_seq=2 ttl=64 time=0.037 ms
^C
```

```
#ping desde contenedor 2 hacia el contenedor 1
[root@d2-bdd-proy-arc /]# ping h1-bdd-proy-jrc.fi.unam
PING h1-bdd-proy-jrc.fi.unam (172.20.0.21) 56(84) bytes of data.
64 bytes from h1-bdd-proy-jrc.fi.unam (172.20.0.21): icmp_seq=1 ttl=64 time=0.054 ms
64 bytes from h1-bdd-proy-jrc.fi.unam (172.20.0.21): icmp_seq=2 ttl=64 time=0.088 ms
^C
```

- D. Actualizar el archivo `/etc/profile.d/99-custom-env.sh` con el nombre del host de cada contenedor

```
#nombre de host en el contenedor 1
export ORACLE_HOSTNAME=h1-bdd-proy-jrc
```

```
#nombre de host en el contenedor 2
export ORACLE_HOSTNAME=h2-bdd-proy-arc
```

- E. Actualizar los nombres de host en el archivo `$ORACLE_HOME/network/admin/listener.ora`

```
#Configuración en el contenedor 1
LISTENER =
  (DESCRIPTION_LIST =
    (DESCRIPTION =
      (ADDRESS = (PROTOCOL = TCP)(HOST = h1-bdd-proy-jrc.fi.unam)(PORT = 1521))
      (ADDRESS = (PROTOCOL = IPC)(KEY = EXTPROC1521))
    )
  )
#Configuración en el contenedor 2
LISTENER =
  (DESCRIPTION_LIST =
    (DESCRIPTION =
      (ADDRESS = (PROTOCOL = TCP)(HOST = h2-bdd-proy-arc.fi.unam)(PORT = 1521))
      (ADDRESS = (PROTOCOL = IPC)(KEY = EXTPROC1521))
    )
  )
```

- F. Configuración del archivo `tnsnames.ora`. Asegurarse de actualizar y agregar los siguientes alias de servicios en cada contenedor para realizar conexiones hacia cualquiera de las 4 PDBs. Actualizar las iniciales según corresponda.

- `FREE` Alias que apunta a `cdb$root` del contenedor 1
- `<INICIALES_INTEGRANTE1>BDD_S1` Alias empleado en el semestre
- `<INICIALES_INTEGRANTE1>BDD_S2` Alias empleado en el semestre
- `<INICIALES_INTEGRANTE2>BDD_S1` Alias que permite conectarse a una nueva PDB
- `<INICIALES_INTEGRANTE2>BDD_S2` Alias que permite conectarse a una nueva PDB

El siguiente ejemplo muestra los cambios realizados al archivo `tnsnames.ora` en el contenedor 1. Realizar una configuración equivalente en el contenedor 2.

#Alias para Las PDBs del integrante 1

```
FREE =
  (DESCRIPTION =
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST = h1-bdd-proy-jrc.fi.unam)(PORT = 1521))
    )
    (CONNECT_DATA =
      (SERVER = DEDICATED)
      (SERVICE_NAME = free.fi.unam)
    )
  )
```

```
JRCBDD_S1 =
  (DESCRIPTION =
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST = h1-bdd-proy-jrc.fi.unam)(PORT = 1521))
    )
    (CONNECT_DATA =
      (SERVICE_NAME = jrcbdd_s1.fi.unam)
    )
  )
```

```
JRCBDD_S2 =
  (DESCRIPTION =
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST = h1-bdd-proy-jrc.fi.unam)(PORT = 1521))
    )
    (CONNECT_DATA =
      (SERVICE_NAME = jrcbdd_s2.fi.unam)
    )
  )
```

#Se agregan Los 2 siguientes alias para Las PDBs del contenedor 2

```
ARCBDD_S1 =
  (DESCRIPTION =
    (ADDRESS_LIST =
      (ADDRESS = (PROTOCOL = TCP)(HOST = h2-bdd-proy-arc.fi.unam)(PORT = 1521))
    )
    (CONNECT_DATA =
      (SERVICE_NAME = arcbdd_s1.fi.unam)
    )
  )
```

```
ARCBDD_S2 =
  (DESCRIPTION =
    (ADDRESS_LIST =
```

```

        (ADDRESS = (PROTOCOL = TCP)(HOST = h2-bdd-proy-arc.fi.unam)(PORT = 1521))
    )
    (CONNECT_DATA =
        (SERVICE_NAME = arcbdd_s2.fi.unam)
    )
)
#No olvidar actualizar el nombre de host, en este caso para el host del contenedor 1
LISTENER_FREE =
    (ADDRESS = (PROTOCOL = TCP)(HOST = h1-bdd-proy-jrc.fi.unam)(PORT = 1521))

```

- G. Debido a que el contenedor 2 es una réplica del contenedor 1, se deberán eliminar las 2 PDBs y crear 2 nuevas PDBs con las iniciales del integrante 2. Para ello, iniciar la CDB en el contenedor 2, como usuario `sys`, borrar las PDBs existentes

```

--en el contenedor 2
alter pluggable database jrcbdd_s1 close;
drop pluggable database jrcbdd_s1 including datafiles;
alter pluggable database jrcbdd_s2 close;
drop pluggable database jrcbdd_s2 including datafiles;

```

- H. Eliminar directorios de las PDBs eliminadas. Ejecutar en el s.o. con usuario `oracle`. Antes de eliminar, comprobar que los 2 directorios estén vacíos.

```

cd /opt/oracle/oradata/FREE
rm -rf jrcbdd_s1 jrcbdd_s2

```

- I. Autenticar como usuario `sys` en `cbd$root`, crear 2 nuevas PDBs en el contenedor 2 con las iniciales del integrante 2. Se crea una nueva PDB a partir de `pdb$seed`.

```

--crea arcbdd_s1
create pluggable database arcbdd_s1
  admin user admin identified by admin
  file_name_convert = (
    '/opt/oracle/oradata/FREE/pdbseed',
    '/opt/oracle/oradata/FREE/arcdbdd_s1'
  );
--crea arcbdd_s2
create pluggable database arcbdd_s2
  admin user admin identified by admin
  file_name_convert = (
    '/opt/oracle/oradata/FREE/pdbseed',
    '/opt/oracle/oradata/FREE/arcdbdd_s2'
  );

```

J. Abrir las PDBs de forma permanente

```
alter pluggable database all open;
alter pluggable database all save state;
```

- K. Crear tablespace `users` en las nuevas PDBs. Al crear nuevas PDBs a partir de `pdb$seed`, el tablespace `users` empleado para almacenar los datos de los usuarios no se incluye, por lo tanto, se debe crear de forma manual. Ajustar los nombres de las PDBs según corresponda

```
--para arcbdd_s1
connect sys@arcbdd_s1 as sysdba
create tablespace users
datafile '/opt/oracle/oradata/FREE/arcbdd_s1/users01.dbf' size 100m
autoextend on next 10m maxsize 11g
extent management local
segment space management auto;
--asignar al tablespace users como el tablespace por defecto
alter database default tablespace users;

--para arcbdd_s2
connect sys@arcbdd_s2 as sysdba
create tablespace users
datafile '/opt/oracle/oradata/FREE/arcbdd_s2/users01.dbf' size 100m
autoextend on next 10m maxsize 11g
extent management local
segment space management auto;
--asignar al tablespace users como el tablespace por defecto
alter database default tablespace users;
```

- L. Cambiar el tablespace por defecto para cada una de las nuevas PDBs para que hagan uso del tablespace `users` creado anteriormente
- M. Para verificar la correcta configuración y conexión bidireccional, ejecutar las siguientes instrucciones en ambos contenedores Docker. Ajustar las iniciales, asegurarse de haber iniciado las 2 CDBs, y sus respectivos listeners.

```
#cdb$root en contenedor actual
sqlplus sys as sysdba
#PDBs en contenedor 1
sqlplus sys@jrcbdd_s1 as sysdba
sqlplus sys@jrcbdd_s2 as sysdba
#PDBs en contenedor 2
```

```
sqlplus sys@arcbdd_s1 as sysdba  
sqlplus sys@arcbdd_s2 as sysdba
```

1.2. Creación de objetos en la BDD

1.2.1. Reglas generales

Antes de iniciar con la creación y ejecución de scripts SQL, considerar las siguientes reglas:

- Los scripts NO deberán ser creados con el usuario `oracle` del sistema operativo.
- Los scripts NO deberán ser ejecutados con el usuario `oracle` del sistema operativo.
- Los scripts NO deberán ser ejecutados con el usuario `SYS` o `SYSTEM` a menos que se trate de sentencias que lo requieran.
- Todos los scripts deberán hacer uso de la sentencia `connect` para cambiarse de usuario o de PDB. Lo anterior implica que al entrar a SQL *Plus se debe emplear la instrucción `sqlplus /nolog`.

Omitir cualquiera de estas reglas causará menos puntos en la calificación.

Cabe mencionar que ambos integrantes deben tener el ambiente completo y funcional. Se recomienda emplear un repositorio privado de control de versiones como GitHub para poder intercambiar scripts. En la entrega ambos integrantes deberán contar con su base de datos distribuida funcionando.

1.2.2. Creación de usuarios

Crear un solo script SQL `s-01-ilap-usuario.sql` El Script deberá realizar las siguientes acciones:

- Crear un usuario llamado `ilap_bdd` en caso de no existir
- Otorgar privilegios para crear sesión, tablas, secuencias, procedimientos, vistas, sinónimos, y ligas. No otorgar más de los permisos requeridos.

Ejemplo

```
--@Autor:          Jorge A. Rodríguez C  
--@Fecha creación: dd/mm/yyyy  
--@Descripción:     Eliminación y creación del usuario.
```

```
Prompt Creando al usuario ilap_bdd  
drop user if exists ilap_bdd;  
--Completar
```

- No olvidar el encabezado a cada script como se muestra en la figura anterior.

- Crear un script llamado `s-01-ilap-main-usuario.sql`. Este script deberá conectarse a los 4 nodos y ejecutar el script anterior.

Ejemplo

```
--@Autor:          Jorge A. Rodríguez C
--@Fecha creación: dd/mm/yyyy
--@Descripción:    Creación de usuarios en Los 4 nodos

clear screen
whenever sqlerror exit rollback;
set serveroutput on

Prompt Iniciando creación/eliminación de usuarios.
accept syspass char prompt 'Proporcione el password de sys: ' hide

prompt =====
prompt Creando usuario en jrcbdd_s1
prompt =====
connect sys/&&syspass@jrcbdd_s1 as sysdba
@s-01-ilap-usuario.sql

prompt =====
prompt Creando usuario en jrcbdd_s2
prompt =====
connect sys/&&syspass@jrcbdd_s2 as sysdba
@s-01-ilap-usuario.sql

prompt =====
prompt Creando usuario en arcbdd_s1
prompt =====
connect sys/&&syspass@arcbdd_s1 as sysdba
@s-01-ilap-usuario.sql

prompt =====
prompt Creando usuario en arcbdd_s2
prompt =====
connect sys/&&syspass@arcbdd_s2 as sysdba
@s-01-ilap-usuario.sql

Prompt Listo!
disconnect
```

1.2.3. Creación de ligas

Generar un script SQL llamado `s-02-ilap-ligas.sql`

- El script deberá contener las instrucciones necesarias para crear las ligas en todos los nodos. No es necesario especificar datos de autenticación ya que el usuario existe en los 4 nodos.
- La comunicación debe ser bidireccional por lo que se requieren crear 3 ligas por nodo, 12 en total.
- Emplear el nombre global de cada PDB como nombre de la liga:

Ejemplo

```
--@Autor:          Jorge A. Rodríguez C
--@Fecha creación:
--@Descripción:    Creación de ligas en los 4 nodos.
clear screen
whenever sqlerror exit rollback;
Prompt =====
Prompt Creando ligas en jrcbdd_s1
Prompt =====

Prompt Creando ligas en jrcbdd_s1
connect ilap_bdd/ilap_bdd@jrcbdd_s1
-- PDB Local
create database link jrcbdd_s2.fi.unam using 'JRCBDD_S2';
--PDB remotas
create database link arcbdd_s1.fi.unam using 'ARCBDD_S1';
Create database link arcbdd_s2.fi.unam using 'ARCBDD_S2';

--completar

Prompt Listo!
```

1.2.4. Creación de fragmentos

El siguiente punto es la creación de todas las tablas que representan los fragmentos en cada sitio. Para ello, generar un archivo por cada PDB llamado `s-03-ilap-<pdb>-ddl.sql`. Ejemplo: `s-03-ilap-jrc-s1-ddl.sql`. Considerar los siguientes puntos.

- Se recomienda emplear Er/Studio para generar el código SQL.
- Hacer copias del modelo global y realizar los cambios correspondientes empleando la tabla de fragmentos generada en el documento anterior.
- Las tablas se deberán crear en orden de dependencias.
- Cada tabla deberá contener la definición de sus restricciones (restricciones de llave primaria, restricciones de integridad: `NOT NULL`, restricciones de referencia).
- Debido al diseño de fragmentación, algunas restricciones de referencia incluidas en el script anterior no se podrán verificar de manera automática. Las únicas

referencias a conservar son las llaves foráneas que provienen de catálogos, las fragmentaciones horizontales derivadas, y algunas fragmentaciones verticales.

- Para identificar las restricciones que se deben conservar, revisar las reglas de fragmentación, y llenar la siguiente tabla.

Restricciones de referencia que se conservan.

Nombre de la tabla Padre	Nombre de la tabla hija	Sitio

- Las restricciones de referencia que no pueden conservarse deberán ser eliminadas desde ER -Studio conservando únicamente el campo en la tabla hija.
 - Para eliminar una restricción de referencia en ER-Studio sin eliminar el campo en la tabla hija, seleccionar la relación, presionar “Supr” o en su defecto hacer clic derecho y seleccionar “Delete Relationship”.
 - Aparecerá una ventana que solicita la confirmación de la eliminación.
 - Observar que el campo “Make Foreign Keys Native” debe ser habilitado para que el campo se conserve en la tabla hija y solo se elimine la restricción de referencia.



- Crear un script llamado `s-03-ilap-main-ddl.sql`. El script deberá conectarse a cada PDB e invocar a los 4 archivos creados anteriormente:

Ejemplo

```
--@Autor:          Jorge A. Rodríguez C
--@Fecha creación: dd/mm/yyyy
--@Descripción:    Creación de fragmentos en los 4 nodos
clear screen
whenever sqlerror exit rollback;

prompt =====
prompt Creando fragmentos para jrcbdd_s1
prompt =====
connect ilap_bdd/ilap_bdd@jrcbdd_s1
@s-03-ilap-jrc-s1-ddl.sql
```

```
prompt =====
prompt Creando fragmentos para jrcbdd_s2
prompt =====
connect ilap_bdd/ilap_bdd@jrcbdd_s2
@s-03-ilap-jrc-s2-ddl.sql

prompt =====
prompt Creando fragmentos para arcbdd_s1
prompt =====
connect ilap_bdd/ilap_bdd@arcbdd_s1
@s-03-ilap-arc-s1-ddl.sql

prompt =====
prompt Creando fragmentos para arcbdd_s2
prompt =====
connect ilap_bdd/ilap_bdd@arcbdd_s2
@s-03-ilap-arc-s2-ddl.sql

Prompt Listo!
```

Ejecutar el script.
Continuar con la siguiente parte del proyecto.